

BLAST (Basic Local Alignment Search Tool)

BLAST is one of the most widely used tools in bioinformatics.

It helps researchers **identify similar sequences**, **predict gene function**, and **find evolutionary relationships**.

Almost every bioinformatics role expects basic BLAST knowledge.

What does BLAST do?

BLAST compares a **query sequence** (DNA or protein) against a **database of known sequences** and finds **regions of local similarity**.

It answers questions like:

- Is my sequence already known?
- What gene/protein is this?
- Does it have homologs in other organisms?

Sequence Similarity

Sequence similarity means how much two biological sequences resemble each other at the nucleotide or amino acid level.

BLAST calculates similarity based on:

- Matches
- Mismatches
- Gaps
- Alignment scores

Higher similarity usually indicates **functional or evolutionary relatedness**.

Homologous Genes

Homologous genes are genes that share a **common evolutionary origin**.

Types of homologs:

- **Orthologs**: Same gene in different species (same function)
- **Paralogs**: Gene duplication within the same species (may have different functions)

BLAST helps identify homologous genes by detecting **significant sequence similarity**.

How BLAST Works (Conceptual Steps)

1. Takes a query sequence
2. Breaks it into small words (k-mers)
3. Searches databases for similar words

4. Extends matches to form alignments
5. Scores and ranks results
6. Reports statistically significant hits

Important BLAST Output Terms

Term	Meaning
Score	Alignment quality
E-value	Probability of random match (lower is better)
Identity (%)	Exact matches
Query coverage	How much of query is aligned
Gaps	Insertions/deletions

E-value < **1e-5** is usually considered significant.

BLAST Tools

NCBI BLAST (Online)

- Web-based
- Easy to use
- No installation required
- Best for beginners and quick analysis

Used when:

- Learning BLAST
- Small number of sequences
- Quick annotation

Local BLAST (Future skill)

- Installed on local machine
- Faster for large datasets
- Used in research labs and industry pipelines

Used when:

- Large-scale analysis
- Automation
- High-throughput projects

BLAST Types (Basic awareness)

BLAST Type	Query	Database
blastn	DNA	DNA

BLAST Type	Query	Database
blastp	Protein	Protein
blastx	DNA → Protein	Protein
tblastn	Protein	DNA
tblastx	DNA → Protein	DNA → Protein

Using BLAST in Biopython

```
from Bio.Blast import NCBIWWW
```

What this does:

It allows you to run **online BLAST searches directly from Python** using NCBI servers.

Requires:

- Internet connection
- Small queries only (NCBI limits apply)

Example (blastn using Biopython)

```
from Bio.Blast import NCBIWWW

result_handle = NCBIWWW.qblast(
    program="blastn",
    database="nt",
    sequence="ATGCGTACGTAGCTAG"
)

with open("blast_result.xml", "w") as out:
    out.write(result_handle.read())
```

Result is saved in **XML format**

XML can be parsed later for hits, scores, and annotations

Why BLAST is job-critical?

BLAST is used in:

- Gene identification
- Functional annotation
- Variant analysis
- Comparative genomics
- Drug target discovery

If you know BLAST well, you are **industry-relevant**.

Parsing BLAST XML with `Bio.Blast.NCBIXML`

BLAST results are usually returned in **XML format**, which is:

- Machine-readable
- Rich in details (scores, identities, E-values, alignments)

To extract useful info, we **parse XML**.

BLAST XML Structure (Concept)

BLAST XML contains:

- One or more **BLAST records**
- Each record has multiple **alignments (hits)**
- Each alignment has **HSPs** (High Scoring Pairs)

Biopython XML Parser

```
from Bio.Blast import NCBIXML
```

Example: Parsing BLAST XML

```
from Bio.Blast import NCBIXML

with open("blast_result.xml") as handle:
    blast_records = NCBIXML.parse(handle)

    for record in blast_records:
        for alignment in record.alignments:
            for hsp in alignment.hsps:
                print("Hit ID:", alignment.hit_id)
                print("Length:", alignment.length)
                print("E-value:", hsp.expect)
                print("Identity:", hsp.identities)
                print("Alignment:")
                print(hsp.query)
                print(hsp.match)
                print(hsp.sbjct)
                print("-" * 40)
```

Important Fields You Should Know

Field	Meaning
hit_id	Sequence ID in database
hit_def	Description
identities	Exact matches
align_length	Alignment length
expect	E-value
score	Alignment score

Interview tip: **HSP = best local alignment region**

Local BLAST Installation (Conceptual)

Running BLAST **on your own computer** instead of NCBI servers.

Why use Local BLAST?

Online BLAST	Local BLAST
Easy	Fast
Limited	No limits
Manual	Automated
Small data	Large datasets

Basic Steps (Awareness level)

1. Download **BLAST+** from NCBI
2. Install on system
3. Download databases (nt, nr, swissprot)
4. Run BLAST via command line

Example command:

```
blastn -query query.fasta -db nt -out result.txt
```

You are **not expected to memorize commands** as a beginner, but concept is important.

Filtering BLAST Results

BLAST gives **many hits**, but not all are meaningful.

Common Filtering Criteria

Filter	Why
E-value	Remove random matches
Identity %	Keep high similarity
Query coverage	Ensure sufficient alignment
Alignment length	Avoid tiny matches

Example: Filter by E-value

```
if hsp.expect < 1e-5:  
    print("Significant hit")
```

Example: Filter by Identity %

```
identity_percent = (hsp.identities / hsp.align_length) * 100
```

```
if identity_percent > 70:
    print("High similarity hit")
```

Industry standard: **E-value** < **1e-5**

Protein BLAST & BLOSUM Matrices

Why Protein BLAST is different?

Protein sequences:

- Tolerate substitutions
- Evolution is slower
- Chemical similarity matters

What is BLOSUM?

BLOSUM (BLOcks SUBstitution Matrix) assigns scores based on:

- How frequently amino acids substitute in evolution

Example:

- Leucine ↔ Isoleucine → high score
- Glycine ↔ Tryptophan → low score

Common BLOSUM Matrices

Matrix	Use
BLOSUM62	General purpose (default)
BLOSUM80	Closely related proteins
BLOSUM45	Distant proteins

Protein BLAST with Biopython

```
from Bio.Blast import NCBIWWW

result_handle = NCBIWWW.qblast(
    program="blastp",
    database="nr",
    sequence="MVLSPADKTNVKAAWGKVGAHAGEYGA"
)
```

BLASTP automatically uses **BLOSUM matrix** (usually BLOSUM62)

PROJECT 1: Unknown Gene Identification & Functional Annotation Pipeline

Problem Statement

You are given an **unknown DNA sequence**, and you need to determine:

- What gene is this?
- Which organism does it belong to?
- What is its biological function?

What Problem Does This Solve?

In real-world biology:

- Sequencing experiments often produce unknown fragments.
- Researchers must identify those sequences.
- Clinical labs need to identify mutations or unknown variants.
- Genome projects require automated annotation.

This project solves:

“Given an unknown DNA sequence, identify its gene and predict its biological function using sequence similarity.”

Tools & Concepts Used

- Biopython
- NCBI BLAST
- BLAST XML parsing
- FASTA parsing
- GenBank parsing
- Sequence annotation extraction

Workflow

Input: DNA sequence (FASTA)

```
↓ Perform BLAST search (blastn)
↓ Save results in XML format
↓ Parse XML using Biopython
↓ Filter hits (E-value < 1e-5)
↓ Select top significant hit
↓ Extract accession ID
↓ Fetch GenBank record using Entrez
↓ Extract features (gene, CDS, product)
```

Output:

```
- Gene name
- Organism
- Functional description
- Identity %
- Query coverage
```

Skills Demonstrated

- Running BLAST programmatically
- Parsing structured XML output
- Filtering biologically significant hits
- Using accession IDs to retrieve records
- Extracting annotated gene features

PROJECT 2: Comparative Gene Analysis & Phylogenetic Study

Problem Statement

You want to study how a specific gene (e.g., BRCA1) varies across species:

- Human
- Mouse
- Chimpanzee
- Zebrafish

And answer:

- How similar are these sequences?
- Which species are more closely related?
- Which regions of the gene are conserved?

What Problem Does This Solve?

This project addresses real research questions such as:

- How conserved is a gene across species?
- Can model organisms (e.g., mouse) represent human genes?
- What does sequence conservation reveal about evolution?

This project solves:

“Analyze cross-species conservation and evolutionary relationships of a gene using multiple sequence alignment and phylogenetics.”

Tools & Concepts Used

- Biopython
- Entrez (sequence download via accession ID)
- FASTA handling
- Multiple Sequence Alignment (MUSCLE or ClustalW)
- Phylogenetic tree construction
- Tree visualization

Workflow

Input: Gene name (e.g., BRCA1)

- Aligned sequences
- Similarity comparison
- Phylogenetic tree
- Evolutionary interpretation

PROJECT 1 SOLUTION

Unknown Gene Identification & Functional Annotation Pipeline

What This Code Does

- Takes a DNA sequence
- Runs BLAST (blastn)
- Parses XML
- Filters best hit
- Extracts accession ID
- Fetches GenBank record
- Extracts gene + product info

Step 1: Required Imports

```
from Bio.Blast import NCBIWWW
from Bio.Blast import NCBIXML
from Bio import Entrez
from Bio import SeqIO
```

Step 2: Set Email (Required for Entrez)

```
Entrez.email = "your_email@example.com"
```

Step 3: Input DNA Sequence

```
sequence = "ATGCGTACGTAGCTAGCTAGCTAGCTAG"
```

Step 4: Run BLAST

```
print("Running BLAST...")
result_handle = NCBIWWW.qblast(
    program="blastn",
    database="nt",
    sequence=sequence
)
```

```
with open("blast_result.xml", "w") as out:
    out.write(result_handle.read())
```

Step 5: Parse BLAST XML

```
with open("blast_result.xml") as handle:
    blast_record = NCBIXML.read(handle)

print("Total hits:", len(blast_record.alignments))
```

Step 6: Select Best Significant Hit

```
best_hit = None

for alignment in blast_record.alignments:
    for hsp in alignment.hsps:
        if hsp.expect < 1e-5:
            best_hit = alignment
            break
    if best_hit:
        break

if best_hit:
    print("Best Hit:", best_hit.hit_def)
    accession = best_hit.accession
    print("Accession:", accession)
else:
    print("No significant hit found.")
```

Step 7: Fetch GenBank Record

```
if best_hit:
    handle = Entrez.efetch(
        db="nucleotide",
        id=accession,
        rettype="gb",
        retmode="text"
    )

    record = SeqIO.read(handle, "genbank")

    print("Organism:", record.annotations["organism"])
```

Step 8: Extract Gene & Product

```
for feature in record.features:
    if feature.type == "CDS":
        gene = feature.qualifiers.get("gene", ["N/A"])[0]
        product = feature.qualifiers.get("product", ["N/A"])[0]

        print("Gene:", gene)
        print("Product:", product)
```

Final Output Example

```
Best Hit: BRCA1 DNA repair associated [Homo sapiens]
Accession: NM_007294
Organism: Homo sapiens
```

Gene: BRCA1
Product: breast cancer type 1 susceptibility protein

PROJECT 2 SOLUTION

Comparative Gene Analysis & Phylogenetic Study

What This Code Does

Fetch BRCA1 from multiple species
Save FASTA
Perform MSA (MUSCLE required)
Build phylogenetic tree

Step 1: Imports

```
from Bio import Entrez, SeqIO, Phylo
from Bio.Align.Applications import MuscleCommandline
import os
```

Step 2: Set Email

```
Entrez.email = "your_email@example.com"
```

Step 3: Fetch Sequences from NCBI

Example accession IDs (replace if needed):

```
accessions = [
    "NM_007294",    # Human BRCA1
    "NM_009764",    # Mouse BRCA1
]

with open("brca1_sequences.fasta", "w") as out:
    for acc in accessions:
        handle = Entrez.efetch(
            db="nucleotide",
            id=acc,
            rettype="fasta",
            retmode="text"
        )
        seq_record = SeqIO.read(handle, "fasta")
        SeqIO.write(seq_record, out, "fasta")
```

Step 4: Perform Multiple Sequence Alignment (MUSCLE)

MUSCLE must be installed.

```
muscle_exe = "muscle"    # path if needed
input_file = "brca1_sequences.fasta"
output_file = "aligned.fasta"

muscle_cline = MuscleCommandline(
    muscle_exe,
```

```
        input=input_file,
        out=output_file
    )

    stdout, stderr = muscle_cline()
```

Step 5: Build Phylogenetic Tree

```
from Bio.Phylo.TreeConstruction import DistanceCalculator,
DistanceTreeConstructor
from Bio import AlignIO

alignment = AlignIO.read("aligned.fasta", "fasta")

calculator = DistanceCalculator("identity")
dm = calculator.get_distance(alignment)

constructor = DistanceTreeConstructor()
tree = constructor.nj(dm)

Phylo.draw(tree)
```

Output

Multiple sequence alignment
Distance matrix
Neighbor-Joining phylogenetic tree